متغیر شرط احضار می‌کند، ReentrantLock مرتبط را رها می‌سازد که با این کار، نخ دیگر اجازه می‌یابد تا قفل انحصار متقابل را کسب کند. با تشابهی، موقعی که ()signal احضار می‌شود، تنها متغیر شرط سیگنال داده می‌شود؛ قفل با احضار ()unlock رها می‌گردد.

```
/* threadNumber is the thread that wishes to do some work */
public void doWork(int threadNumber)
{
        lock.lock();
        try {
                /**
                * If it's not my turn, then wait
                * until I'm signaled.
                */
                if (threadNumber != turn)
                        condVars[threadNumber].await();
                /**
                * Do some work for awhile ...
                */
                /**
                * Now signal to the next thread.
                */
                turn = (turn + 1) % 5;
                condVars[turn].signal();
        }
        catch (InterruptedException ie) { }
        finally {
                lock.unlock();
        }
}
```

شکل ۷-۱۳ مثال استفاده از متغیرهای شرط جاوا.

۷-۵ رویکردهای جایگزین

با پیدایش سیستم‌های چندهسته‌ای، فشار مضاعفی بر توسعه‌ی کاربردهای چندنخی است تا از مزیت هسته‌های پردازشی متعدد بهره ببرند. با این وجود، برنامه‌های همروند، خطر رو به افزایش شرایط مسابقه و خطرات عاقبت به‌خیری، همچون بن‌بست را پیش می‌کشند. در سابق، تکنیک‌هایی از قبیل قفل‌های انحصار متقابل، سمافورها و مانیتورها برای رفع این نگرانی‌ها استفاده می‌شد، اما با افزایش تعداد هسته‌های پردازشی، طراحی کاربردهای چندنخی که عاری از شرایط مسابقه و بن‌بست باشند به صورت فزاینده دشوارتر می‌شود. در این بخش، امکانات گوناگونی را مرور می‌کنیم که در هر دوی زبان‌های برنامه‌نویسی و سخت‌افزار که طراحی کاربردهای همروند نخ ایمن را پشتیبانی می‌کنند، فراهم شده است.

۷-۵-۱ حافظه‌ی تراکنشی

زیاد اتفاق می‌افتد که در علوم کامپیوتر، ایده‌های یک بخش از تحقیقات می‌توانند برای حل مسائلی در سایر بخش‌ها به‌کار گرفته شوند. برای مثال، مفهوم **حافظه‌ی تراکنشی**[1] به رغم این که ریشه در تئوری پایگاه‌داده‌ها دارد، اما یک راهبرد برای همگامی پردازه فراهم می‌کند. **تراکنش حافظه** دنباله‌ای از عملیات خواندن-نوشتن حافظه است که یکپارچه هستند. اگر تمامی عملیات یک تراکنش تکمیل شوند، تراکنش حافظه نهایی می‌شود. در غیر این صورت، بایستی عملیات رها شده و به عقب کشیده شوند. منافع حافظه‌ی تراکنشی می‌تواند از طریق افزودن امکاناتی به یک زبان برنامه‌نویسی حاصل شود.

یک مثال را در نظر بگیرید. فرض کنید یک تابع update() داریم که داده‌های مشترکی را تغییر می‌دهد. این تابع در سابق با استفاده از قفل‌ها (یا سمافورها) به صورت زیر نوشته می‌شد:

```
void update ()
{
        acquire();
        /* modify shared data */
        release();
}
```

با این وجود، استفاده از راهکارهای همگامی از قبیل قفل‌های انحصار متقابل و سمافورها، بسیاری از مسائل بالقوه از جمله بن‌بست را به بار می‌آورد. به علاوه، قفل‌گذاری سابق با افزایش تعداد نخ‌ها از توسعه‌پذیری خوبی برخوردار نیست.

به عنوان یک روش جدید جایگزین روش‌های سابق، می‌توان امکانات جدیدی را که از حافظه‌ی تراکنشی بهره می‌برند به یک زبان برنامه‌نویسی اضافه کرد. در مثالی که ارائه کردیم، فرض کنید بنای atomic{S} را اضافه کنیم که اطمینان می‌دهد عملیات S به صورت یک تراکنش اجرا می‌شود. به این ترتیب، می‌توانیم متد update() را به صورت زیر بازنویسی کنیم:

```
void update ()
{
        atomic {
                /* modify shared data */
        }
}
```

مزیت استفاده از چنین راهکاری به جای قفل‌ها این است که، سیستم حافظه‌ی تراکنشی—نه برنامه‌نویس—مسؤولیت تضمین یکپارچگی را بر عهده دارد. به علاوه، به خاطر این که قفلی در کار نیست، وقوع بن‌بست ناممکن است. به علاوه، سیستم حافظه‌ی تراکنشی می‌تواند مشخص کند که کدام دستورات در بلاک‌های یکپارچه، همچون دسترسی خواندن همروند متغیر مشترک، می‌توانند به صورت همروند اجرا شوند. البته، این امکان هست که برنامه‌نویس، این شرایط را

1. Transactional Memory

شناسایی و از قفل‌های خواننده-نویسنده استفاده نماید، اما این کار می‌تواند با رشد تعداد نخ‌های یک برنامه‌ی کاربردی فوق‌العاده دشوار شود.

حافظه‌ی تراکنشی می‌تواند به صورت سخت‌افزاری و یا نرم‌افزاری پیاده‌سازی شود. چنان که از نام **حافظه‌ی تراکنشی نرم‌افزاری**[1] (STM) پیداست، منحصراً در نرم‌افزار پیاده‌سازی می‌شود—هیچ سخت‌افزار خاصی لازم ندارد. STM با درج اسباب کد در بلاک‌های تراکنش کار می‌کند. این کد توسط یک کامپایلر درج می‌شود و هر یک از تراکنش‌ها را با بررسی این که، کدام دستورات می‌توانند اجرای همروند داشته باشند و کجا لازم است از قفل‌گذاری سطح پایین استفاده شود، مدیریت می‌کند. **حافظه‌ی تراکنشی سخت‌افزاری**[2] (HTM) از سلسله‌مراتب کَش سخت‌افزاری و پروتکل‌های سازگاری کَش استفاده می‌کند تا برخوردهای حاصل از داده‌های مشترک مقیم کَش‌های پردازنده‌های مجزا را مدیریت و رفع نماید. HTM به هیچ اسباب کدی نیاز ندارد و لذا سربار آن از STM کمتر است. با این وجود، HTM مستلزم تغییر سلسله‌مراتب کَش و پروتکل‌های سازگاری کَش موجود می‌باشد تا از حافظه‌ی تراکنشی پشتیبانی نمایند.

حافظه‌ی تراکنشی بی آنکه پیاده‌سازی فراگیری داشته باشد چندین سال است که وجود دارد. با این وجود، رشد سیستم‌های چندهسته‌ای و پافشاری آن بر برنامه‌نویسی همروند و موازی، تحقیقات زیادی را در این زمینه توأمان در محافل دانشگاهی و فروشندگان نرم‌افزار تجاری و سخت‌افزار به راه انداخته است.

OpenMP ۲-۵-۷

در بخش ۴-۵-۲، کلیاتی از OpenMP و پشتیبانی آن از برنامه‌نویسی موازی را در یک محیط حافظه‌ی مشترک ارائه کردیم. به خاطر دارید که OpenMP شامل یک مجموعه دایرکتیو کامپایلر و یک API است. هر کدی که به دنبال دایرکتیو کامپایلر #pragma omp parallel قرار می‌گیرد به عنوان یک ناحیه‌ی موازی تلقی شده و توسط تعدادی نخ که برابر تعداد هسته‌های پردازشی سیستم است، انجام می‌شود. مزیت OpenMP (و ابزار مشابه آن) این است که ایجاد و مدیریت نخ در آن، توسط کتابخانه‌ی OpenMP انجام گرفته و مسؤولیت برنامه‌نویسان کاربردی نیست.

به همراه دایرکتیو کامپایلر #pragma omp parallel دایرکتیو کامپایلر #pragma omp critical را فراهم می‌کند که مشخص می‌کند کدی که به دنبال دایرکتیو است یک بخش بحرانی است که در آن، فقط یک نخ می‌تواند در هر لحظه فعال باشد. با این شیوه، OpenMP این تضمین را می‌دهد که نخ‌ها شرایط مسابقه به وجود نمی‌آورند.

به عنوان مثالی از کاربرد دایرکتیو کامپایلر بخش بحرانی، نخست فرض کنید که متغیر مشترک counter می‌تواند در تابع ()update به صورت زیر تغییر یابد:

```
void update(int value)
{
```

1. Software transactional memory
2. Hardware transactional memory

```
        counter += value;
}
```

اگر تابع ()update بتواند بخشی از ناحیه‌ی موازی بوده (یا از ناحیه‌ی موازی احضار شود)، یک شرط مسابقه بر روی متغیر counter امکان‌پذیر است.

دایرکتیو کامپایلر بخش بحرانی می‌تواند برای رفع این شرط مسابقه استفاده شود و کد آن به صورت زیر است:

```
void update(int value)
{
        #pragma omp critical
        {
                counter += value;
        }
}
```

دایرکتیو کامپایلر بخش بحرانی بیشتر شبیه یک سمافور دودویی یا قفل انحصار متقابل رفتار می‌کند، با تضمین این که فقط یک نخ می‌تواند در هر لحظه در بخش بحرانی فعال باشد. اگر یک نخ، موقعی که نخ دیگری فعلاً در بخش بحرانی فعال است (یعنی، ***مالکیت*** بخش را دارد)، اقدام به ورود به بخش بحرانی نماید، نخ فراخواننده مسدود می‌شود تا این که نخ مالک خارج شود. اگر می‌بایست بخش‌های بحرانی متعددی استفاده شوند، می‌توان به هر بخش بحرانی یک نام مجزا داد و یک قانون وضع کرد که بیش از یک نخ نمی‌تواند در یک بخش بحرانی با نام یکسان به صورت همزمان فعال باشد.

یک مزیت استفاده از دایرکتیو کامپایلر بخش بحرانی در OpenMP این است که عموماً استفاده از آن در مقایسه با قفل‌های انحصار متقابل معمول آسان‌تر است. با این همه، عیبی که دارد این است که باز هم بایستی برنامه‌نویسان کاربردی شرایط مسابقه‌ی ممکن را مشخص کنند و به صورت کافی، داده‌های مشترک را با استفاده از دایرکتیو کامپایلر محافظت کنند. به علاوه، چون دایرکتیو کامپایلر بخش بحرانی بیشتر شبیه به یک قفل انحصار متقابل رفتار می‌کند، باز هم موقعی که دو یا چند بخش بحرانی شناسایی می‌شوند، بن‌بست امکان‌پذیر است.

۷-۵-۳ زبان‌های برنامه‌نویسی تابعی

اکثر زبان‌های برنامه‌نویسی معروف–از قبیل C، C++، جاوا و C#–به زبان‌های **امری** (یا **روالی**) معروف هستند. زبان‌های امری برای پیاده‌سازی الگوریتم‌هایی استفاده می‌شوند که مبتنی بر حالت هستند. در این زبان‌ها، جریان الگوریتم برای درستی عملیات آن حیاتی است و حالت، با متغیرها و ساختمان‌داده‌های دیگر نمایش داده می‌شود. البته، حالت برنامه قابل تغییر است، چرا که متغیرها می‌توانند با گذر زمان، مقادیر متفاوتی به خود بگیرند.

با تأکید فعلی بر روی برنامه‌نویسی همروند و موازی سیستم‌های چندهسته‌ای، تمرکز بیشتری روی زبان‌های برنامه‌نویسی **تابعی** است که از یک سبک برنامه‌نویسی بسیار متفاوت با

آنچه که توسط زبان‌های امری عرضه می‌شود، پیروی می‌کنند. تفاوت اساسی میان زبان‌های امری و تابعی در این است که زبان‌های تابعی حالت را نگه نمی‌دارند. یعنی، همین که یک متغیر تعریف شد و یک مقدار گرفت، مقدار آن غیرقابل تغییر است. چون زبان‌های تابعی، حالت قابل تغییر را اجازه نمی‌دهند، نیازی ندارند که نگران ملاحظاتی از قبیل شرایط مسابقه و بن‌بست‌ها باشند. در اصل، اکثر مسائلی که در این فصل مطرح شدند در زبان‌های تابعی وجود ندارند.

زبان‌های تابعی متعددی در حال حاضر استفاده دارند و دو مورد از آنها را در اینجا به اختصار مطرح می‌کنیم: Erlang و Scala. زبان Erlang به خاطر پشتیبانی همروندی و سهولت به‌کارگیری آن در نوشتن برنامه‌هایی که بر روی سیستم‌های موازی اجرا می‌شوند، توجه قابل ملاحظه‌ای را به خود جلب کرده است. Scala یک زبان تابعی است که شئ‌گرا هم هست. در حقیقت، بیشتر نحو Scala مشابه زبان‌های شئ‌گرای فراگیر جاوا و C# می‌باشد.

۷-۶ خلاصه

- مسئله‌های کلاسیک همگامی پردازه شامل مسئله‌های بافر محدود، خوانندگان-نویسندگان، و فیلسوفان خوش‌خوراک هستند. راه‌حل‌های این مسئله‌ها می‌توانند با استفاده از ابزار مطرح شده در فصل ۶، اعم از قفل‌های انحصار متقابل، سمافورها، مانیتورها، و متغیرهای شرط توسعه داده شوند.
- ویندوز به‌منظور پیاده‌سازی ابزار همگامی پردازه از اشیاء اعزام‌کننده در کنار رویدادها استفاده می‌کند.
- لینوکس از رویکردهای مختلفی برای حفاظت در مقابل شرایط مسابقه استفاده می‌کند که عبارتند از: متغیرهای یکپارچه، قفل‌های چرخشی، و قفل‌های انحصار متقابل.
- API پوزیکس، قفل‌های انحصار متقابل، سمافورها، و متغیرهای شرط را فراهم می‌کند. پوزیکس دو شکل شمافور دارد که عبارتند از: با نام و بی‌نام. چندین پردازه‌ی غیرمرتبط می‌توانند به‌راحتی سمافور با نام یکسانی را به‌سادگی با رجوع به نام آن دسترسی نمایند. سمافورهای بی‌نام نمی‌توانند به‌راحتی به‌اشتراک گذاشته شوند، و مستلزم گذاشتن سمافور در یک ناحیه‌ی حافظه‌ی مشترک می‌باشد.
- جاوا دارای یک کتابخانه و API غنی برای همگامی است. ابزار در دسترس شامل مانیتورها (که در سطح زبان فراهم می‌شوند) در کنار قفل‌های چندمدخلی، سمافورها، و متغیرهای شرط (که توسط API پشتیبانی می‌شوند) می‌باشند.
- رویکردهای جایگزین حل مسئله‌ی بخش بحرانی شامل حافظه‌ی تراکنشی، OpenMP، و زبان‌های تابعی است. زبان‌های تابعی جذابیت خاصی دارند، چرا که یک سبک برنامه‌نویسی متفاوتی از زبان‌های روالی را عرضه می‌کنند. برعکس زبان‌های روالی، زبان‌های تابعی حالت نگه نمی‌دارند و لذا عموماً ایمن از شرایط مسابقه و بخش‌های بحرانی هستند.